Step 3 — Rendering: frasi, posizioni dei token, formato degli esempi

Obiettivo

Trasformare ogni riga in un esempio con: - i token del prompt (claim prima del documento, domanda unica); - la posizione `answer_pos` da cui si leggono le lettere (come nel piano binario); - la posizione dell'**ultimo token di ogni frase** del documento (`sent_end_pos`); - i target del verdetto e, dove noti, dell'evidenza.

La stessa funzione deve servire in training e in produzione: una differenza nella segmentazione o nelle posizioni rende la testa inutilizzabile.

1. Formato di un esempio renderizzato

```
{"id": "hover:train:...", "model": "winnow-12b-bf16",
 "input_ids": ["<token del prompt>"], "answer_pos": 1287,
 "letter_ids": [236776, 236799], "target": [0.0, 1.0],
 "sent_end_pos": [212, 248, 301, "…", 1240],
 "sent_char_spans": [[0, 112], [113, 260], "…"],
 "evidence_target": [1, 0, 0, "…", 1],
 "evidence_mask": 1, "evidence_weight": 1.0}
```

- `sent_end_pos[k]`: indice del token che chiude la frase k nel prompt.
- `evidence_mask = 0` per righe negative o senza evidenza nota.
- `sent_char_spans`: servono in produzione per evidenziare sul testo originale.

2. Segmentazione in frasi

Una sola funzione, `finetune/sentences.py`, usata da builder (02a), rendering, valutazione e produzione.

Regola	Perché
Splitter: NLTK `sent_tokenize` (come MiniCheck/FactCG), poi correzioni	standard, veloce
`\n\n` e righe di elenco sono sempre confini	titoli, elenchi, Q/R di AVeriTeC
Righe `Title: …` sono una "frase" a sé	documenti FEVER/HoVer
Frasi > 60 parole spezzate su `;`, `:` o, in ultima istanza, a metà su una virgola	evidenziare meno testo irrilevante
Frasi < 3 parole unite alla successiva	niente evidenze di una parola
Massimo 512 frasi per documento	memoria dell'aggregatore; oltre si segnala

Per le fonti con frasi già date (FEVER `lines`, SciFact `abstract`) si usano quelle; la funzione deve comunque produrre le stesse `sent_char_spans` sul `document` ricostruito.

3. Dalle frasi alle posizioni dei token

```
1. rendering del prompt con la funzione del benchmark (claim prima, domanda unica)
2. nel testo del prompt, trova l'intervallo in cui è scritto il documento
3. mappa ogni sent_char_span del documento sull'intervallo del prompt
     (Winnow: il documento è una stringa JSON escapata → mappa offset originale → offset escapato)
4. tokenizza con return_offsets_mapping=True
5. sent_end_pos[k] = indice dell'ultimo token il cui offset cade dentro la frase k
```

Dettagli che rompono tutto se sbagliati: - **Escaping JSON (Winnow)**: `"` → `\"`, a capo → `\n`, `<` → `\u003c` (`safe_data` in winnow-12b-bf16/benchmark.py). La mappa degli offset va costruita carattere per carattere durante l'escaping, non stimata. - **Token a cavallo** di due frasi (es. `."` unito in un token): si assegna alla frase che contiene l'ultimo carattere non spazio del token. - **Tokenizzazione separata** di prefisso e suffisso (Winnow, ../fine_tuning/03_rendering_formato.md §2): gli offset vanno ricalcolati sui pezzi effettivamente tokenizzati.

4. Documenti lunghi

Niente chunking (00a §5): il documento passa intero.

Fase	Budget	Cosa fare oltre il budget
Training	8.192 token (16k se la memoria lo permette con batch 1)	righe positive con evidenza: si tagliano frasi **non evidenza** lontane dall'evidenza finché si rientra (le etichette restano valide); righe negative: si taglia la coda; contare i casi
Produzione	contesto del modello (decine di migliaia di token)	oltre il contesto: fuori perimetro, si segnala

Il taglio dei positivi è più facile che nel piano binario: sapendo dove sta l'evidenza, si possono togliere solo frasi che non ne fanno parte.

5. Test obbligatori

Test	Cosa verifica
Ricopertura	le `sent_char_spans` coprono il documento senza buchi (a parte spazi) né sovrapposizioni
Posizioni	per 200 righe, il testo decodificato dal token `sent_end_pos[k]` è la fine della frase k (stampa e confronto automatico)
Escaping	documenti con virgolette, a capo, <, emoji, testo non latino: posizioni corrette
Identità del prompt	a parte l'ordine dei campi, prompt identico a quello del benchmark
Coerenza training/produzione	la funzione di produzione e quella di training danno lo stesso `sent_end_pos` sugli stessi input

Criteri per passare allo step successivo

- Test di §5 superati.
- Statistiche: frasi per documento (p50/p95), token per documento, righe tagliate per fonte.